

**STUDENT NUMBER: ST10479825**

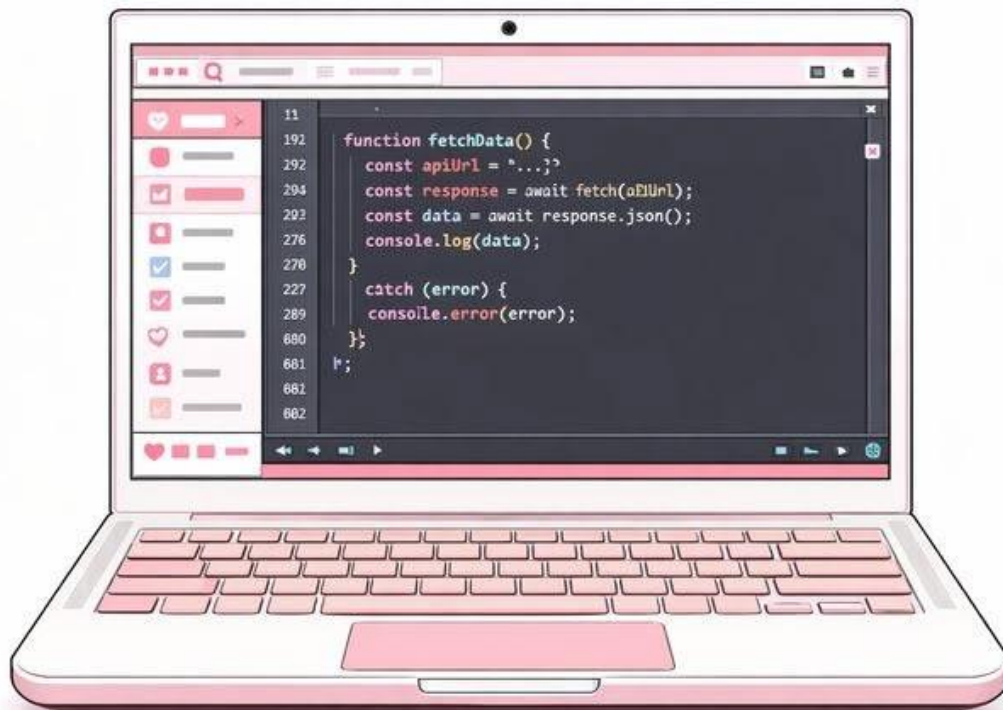
**NAME: THATO**

**SURNAME: MOLAPISANE**

**MODULE: PROGRAMMING**

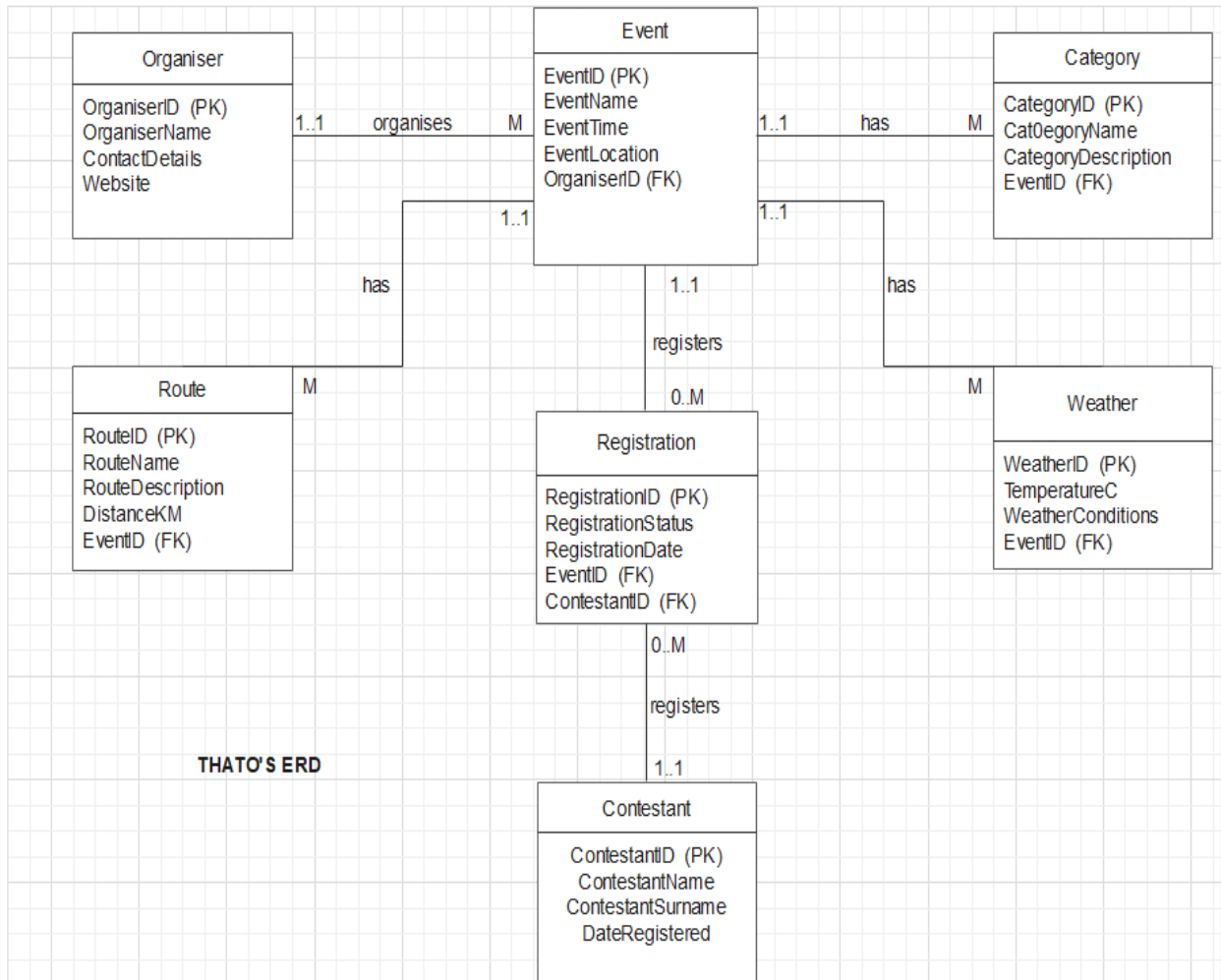
**MODULE CODE: PROG6212**

**POE PART 1**



# Section A

## ERD DIAGRAM



## Section B

| HT<br>TP<br>Me<br>th<br>od | Route              | Desc<br>ription                                                    | Role<br>Required | Request<br>Body                                       | Expected Response                                                                   |
|----------------------------|--------------------|--------------------------------------------------------------------|------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------|
| <b>POST</b>                | /api/auth/register | Registers a new contestant account.                                | None (public)    | { "UserName": "", "UserSurname": "", "Password": "" } | 201 Created - Contestant registered successfully / 400 Bad Request - Missing fields |
| <b>POST</b>                | /api/auth/login    | Authenticates a contestant and returns a token.                    | None (public)    | { "UserName": "", "Password": "" }                    | 200 OK - Token returned / 401 Unauthorized - Invalid credentials                    |
| <b>GET</b>                 | /api/events        | Retrieves all upcoming events with organiser and location details. | Any (logged in)  | None                                                  | 200 OK - List of events / 404 Not Found - No events available                       |
| <b>GET</b>                 | /api/events/{id}   | Returns event details                                              | Any (logged in)  | None                                                  | 200 OK - Event details / 404 Not Found - Event does not exist                       |

|             |                        |                                                   |                   |                                                                                        |                                                                           |
|-------------|------------------------|---------------------------------------------------|-------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------|
|             |                        | ed<br>info<br>for a<br>speci<br>fic<br>event<br>. |                   |                                                                                        |                                                                           |
| <b>POST</b> | /api/events            | Creates a new event under an organiser.           | Admin / Organiser | { "Event Name": "",<br>"EventTime": "",<br>"EventLocation": "",<br>"OrganiserID": "" } | 201 Created – Event added / 400 Bad Request – Invalid data                |
| <b>GET</b>  | /api/categories        | Lists all categories for each event .             | Any (logged in)   | None                                                                                   | 200 OK – Category list / 404 Not Found – No categories                    |
| <b>POST</b> | /api/registration      | Registers a contestant for an event .             | Contestant        | { "EventID": "",<br>"UserID": "",<br>"RegistrationStatus": "" }                        | 201 Created – Registration successful / 409 Conflict – Already registered |
| <b>GET</b>  | /api/registration/{id} | Retrieves registration details for a contestant.  | Contestant        | None                                                                                   | 200 OK – Registration info / 404 Not Found – No record                    |

|            |                        |                                                   |                 |      |                                                              |
|------------|------------------------|---------------------------------------------------|-----------------|------|--------------------------------------------------------------|
| <b>GET</b> | /api/weather/{eventId} | Displays weather conditions for a specific event. | Any (logged in) | None | 200 OK - Weather data / 404 Not Found - No weather record    |
| <b>GET</b> | /api/results/{eventId} | Retrieves race results for an event.              | Any (logged in) | None | 200 OK - Results list / 404 Not Found - No results available |



## **Section C**

```
1  CREATE database RACEDAY;
2  USE RACEDAY;
3
4  CREATE TABLE Organiser (
5      OrganiserID INT PRIMARY KEY IDENTITY (1,1),
6      OrganiserName VARCHAR(100) NOT NULL,
7      ContactDetails VARCHAR(150),
8      Website VARCHAR(150)
9  );
10
11 CREATE TABLE Event (
12     EventID INT PRIMARY KEY IDENTITY (1,1),
13     EventName VARCHAR(100) NOT NULL,
14     EventTime DATETIME NOT NULL,
15     EventLocation VARCHAR(150) NOT NULL,
16     OrganiserID INT,
17     FOREIGN KEY (OrganiserID) REFERENCES Organiser(OrganiserID)
18 );
19
20 CREATE TABLE Category (
21     CategoryID INT PRIMARY KEY IDENTITY (1,1),
22     CategoryName VARCHAR(100) NOT NULL,
23     CategoryDescription VARCHAR(200),
24     EventID INT,
25     FOREIGN KEY (EventID) REFERENCES Event(EventID)
26 );
27
28 CREATE TABLE Route (
29     RouteID INT PRIMARY KEY IDENTITY (1,1),
30     RouteName VARCHAR(100) NOT NULL,
31     RouteDescription VARCHAR(200),
32     DistanceKM DECIMAL(5,2),
33     EventID INT,
34     FOREIGN KEY (EventID) REFERENCES Event(EventID)
35 );
36
```

PROGQUERY....udent (51))\* ✨ ×

```
36
37 CREATE TABLE Weather (
38     WeatherID INT PRIMARY KEY IDENTITY (1,1),
39     TemperatureC DECIMAL(4,1),
40     WeatherConditions VARCHAR(100),
41     EventID INT,
42     FOREIGN KEY (EventID) REFERENCES Event(EventID)
43 );
44
45 CREATE TABLE Contestant (
46     UserID INT PRIMARY KEY IDENTITY (1,1),
47     UserName VARCHAR(100) NOT NULL,
48     UserSurname VARCHAR(100) NOT NULL,
49     DateRegistered DATE NOT NULL
50 );
51
52 CREATE TABLE Registration (
53     RegistrationID INT PRIMARY KEY IDENTITY (1,1),
54     RegistrationStatus VARCHAR(50),
55     RegistrationDate DATE NOT NULL,
56     EventID INT,
57     UserID INT,
58     FOREIGN KEY (EventID) REFERENCES Event(EventID),
59     FOREIGN KEY (UserID) REFERENCES Contestant(UserID)
60 );
61
```

PROGQUERY....udent (51))\* ✨ ×

```
61
62 -- Insert Organisers
63 INSERT INTO Organiser (OrganiserName, ContactDetails, Website)
64 VALUES
65 ('RunSA', 'info@runsa.co.za', 'www.runsa.co.za'),
66 ('Marathon Masters', 'contact@mm.co.za', 'www.mm.co.za');
67
68 -- Insert Events
69 INSERT INTO Event (EventName, EventTime, EventLocation, OrganiserID)
70 VALUES
71 ('Johannesburg City Marathon', '2026-10-15 07:00:00', 'Johannesburg CBD', 1),
72 ('Cape Town Ultra', '2026-11-20 06:30:00', 'Cape Town Stadium', 2);
73
74 -- Insert Categories
75 INSERT INTO Category (CategoryName, CategoryDescription, EventID)
76 VALUES
77 ('10KM Run', 'Short distance for beginners', 1),
78 ('42KM Marathon', 'Full marathon distance', 1),
79 ('21KM Half Marathon', 'Intermediate distance', 2);
80
81 -- Insert Routes
82 INSERT INTO Route (RouteName, RouteDescription, DistanceKM, EventID)
83 VALUES
84 ('CBD Loop', 'Scenic loop through Johannesburg CBD', 10.00, 1),
85 ('Stadium to Waterfront', 'Route from stadium to waterfront', 21.00, 2),
86 ('Mountain Challenge', 'Hilly terrain around Cape Town', 42.00, 2);
87
88 -- Insert Weather
89 INSERT INTO Weather (TemperatureC, WeatherConditions, EventID)
90 VALUES
91 (18.5, 'Sunny', 1),
92 (16.0, 'Cloudy with light wind', 2);
93
```

```

94  -- Insert Contestants
95  INSERT INTO Contestant (UserName, UserSurname, DateRegistered)
96  VALUES
97  ('Thabo', 'Mokoena', '2026-08-01'),
98  ('Lerato', 'Dlamini', '2026-08-05'),
99  ('Sipho', 'Nkosi', '2026-08-10');
100
101  -- Insert Registrations
102  INSERT INTO Registration (RegistrationStatus, RegistrationDate, EventID, UserID)
103  VALUES
104  ('Confirmed', '2026-09-01', 1, 1),
105  ('Pending', '2026-09-02', 1, 2),
106  ('Confirmed', '2026-09-03', 2, 3);
107
108  select * from Organiser;
109  select * from Weather;
110  select * from Contestant;
111  select * from Registration;
112  select * from Route;
113  select * from Event;
114

```

## References

- Biehl, M. (2016). *RESTful API Design (Api-University #3)*. Createspace Independent Publishing Platform. Available at: [https://www.harvard.com/book/restful\\_api\\_design\\_api-university\\_3](https://www.harvard.com/book/restful_api_design_api-university_3) (harvard.com in Bing)
- Chen, P. (1976) 'The Entity-Relationship Model—Toward a Unified View of Data', *ACM Transactions on Database Systems*, 1(1), pp. 9–36. Available at: <http://csc.lsu.edu/news/erd.pdf>
- Enterprise Architecture at Harvard (2020) *Standard API Principles and Practices: Use of the API Gateway and Portal*. Harvard University IT. Available at: <https://enterprisearchitecture.harvard.edu/resource/apigatewayportalstandards04.pdf>
- McFadyen, R. (2002) *Entity-Relationship Modeling: Historical Events, Future Trends, and Lessons Learned*. In: *Software Pioneers: Contributions to Software Engineering*. Springer. Available at: [http://bit.csc.lsu.edu/~chen/pdf/Chen\\_Pioneers.pdf](http://bit.csc.lsu.edu/~chen/pdf/Chen_Pioneers.pdf)
- CS50 (2024) *Lecture 2: Introduction to Databases with SQL*. Harvard University. Available at: <https://cs50.harvard.edu/sql/2024/lectures/2> (cs50.harvard.edu in Bing).
- Wikipedia (2026) *Entity-relationship model*. Available at: [https://en.wikipedia.org/wiki/Entity-relationship\\_model](https://en.wikipedia.org/wiki/Entity-relationship_model) (en.wikipedia.org in Bing)

- IMM Graduate School (2026) *Harvard-Anglia Referencing System Guide*. IMM Graduate School. Available at: [https://imm-gsm.s3.us-east-1.amazonaws.com/eLearn/Home%20Page/Tutorial%20Letters/2026/Semester%202/IMM\\_Graduate\\_School\\_Harvard\\_Referencing\\_Guide\\_2026-S2.pdf](https://imm-gsm.s3.us-east-1.amazonaws.com/eLearn/Home%20Page/Tutorial%20Letters/2026/Semester%202/IMM_Graduate_School_Harvard_Referencing_Guide_2026-S2.pdf)